PROJECT WALKTHROUGH

Employee Attendance & Analytics API

How the FastAPI application, MongoDB collections, attendance rules, and analytics fit together.

Python 3.11+ FastAPI PyMongo MongoDB 6+

Start point: `uvicorn app.main:app --port 8000`
Primary code: `app/main.py`, `app/models/`, `app/services/`, and `app/database.py`

1. Project at a glance

This is a small HTTP service for recording employee attendance and asking questions about it. FastAPI receives and validates HTTP requests, Python applies attendance rules, and MongoDB stores employee and attendance documents. Analytics are computed by MongoDB aggregation pipelines.

Client	**FastAPI route**	**Service logic**	**MongoDB**
Browser, API client, or grader sends HTTP request	Checks inputs and coordinates database/service calls	Converts time, applies rules, builds analytics pipelines	Persists documents, indexes, and executes queries

JSON response
Returns contract fields and epoch-millisecond instants

Two collections

Collection	Purpose	Important fields
`employees`	One document for each employee.	`emp_code` (business identifier), `department`, `shift_start`, `shift_end`, `joined_on`, `created_at`.
`attendance_logs`	At most one attendance record per employee and attendance date.	`emp_code`, `date`, `status`, punch timestamps, derived totals, and correction `history`.

> **Identity:** the API identifies employees by `emp_code`. MongoDB `_id` is internal and is not returned as an API identifier.

Important representations

- Instants arrive in and leave the API as integer epoch milliseconds; MongoDB stores them as UTC BSON datetimes.
- Attendance dates, join dates, and shift times are calendar/time strings, not instants.
- Attendance business dates and shift times use India Standard Time (IST, UTC+05:30).
- `work_hours`, `late_minutes`, `overtime_minutes`, and `half_day` are calculated when writing records. Analytics reads these stored values.

2. What each file is for

File or folder	Responsibility
`app/main.py`	Application entry point: creates the FastAPI app, attaches startup behavior, registers every API route, and coordinates validation and database/service operations.
`app/database.py`	Loads local environment configuration, creates the timezone-aware MongoDB client, selects the database, and defines idempotent collection indexes.
`app/models/schemas.py`	Pydantic request schemas. They validate employee fields, allowed statuses, and strict integer timestamp ranges before route code runs.
`app/services/attendance.py`	Shared attendance rules: UTC/IST conversions, overnight attendance date, late/overtime/work-hour calculations, derived fields, and response serialization.
`app/services/analytics.py`	MongoDB pipeline builders for monthly summaries, department summaries, late ranking, daily trend; also aggregation/explain and query validation helpers.
`app/__init__.py`	Marks `app` as a Python package so imports and the Uvicorn module path resolve.
`app/models/__init__.py`, `app/services/__init__.py`	Mark these folders as Python packages.
`requirements.txt`	Lists runtime libraries installed with pip: FastAPI, Uvicorn, PyMongo, Pydantic, and python-dotenv.
`README.md`	Short setup, run, and package-structure guide.
`openapi.yaml`	Assignment contract: endpoint paths, parameters, schemas, status codes, and business rules. It describes expected behavior; the Python code implements it.
`DATA_MODEL.md`	MongoDB collection/document shapes and sample record explanations.
`sample_seed.py`	Loads the JSON samples into the configured database for local experimentation.
`sample_data/employees.json`, `sample_data/attendance_logs.json`	Small example datasets in MongoDB Extended JSON format.
`PROBLEM_STATEMENT.docx`, `REVIEW.md`, `DECISIONS.md`	Assignment, starter-code defect review, and short explanations of implementation decisions.

The README and request to keep the code structured describe the current modular layout. The assignment's original submit section requested a single application file; the current layout instead separates models, services, and database setup.

3. Startup and local setup

1. Install the listed Python dependencies into a virtual environment.
2. Start MongoDB (local service or Atlas) and set `MONGO_URI` and `MONGO_DB`.
3. Run Uvicorn with module path `app.main:app`.
4. Uvicorn imports `app.main`; that imports the database, schemas, and services.
5. FastAPI's lifespan handler runs `create_indexes()` before serving requests.
6. Use `GET /health` to confirm MongoDB responds; then browse `/docs` for interactive API docs.

```
cd /d C:\Users\hp\Desktop\candidate_kit
.venv\Scripts\python.exe -m pip install -r requirements.txt
set MONGO_URI=mongodb://localhost:27017
set MONGO_DB=attendance_db
.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000 --reload
```

When you use Atlas, set `MONGO_URI` to its connection string instead. Do not share or commit credentials. A `.env` file is optional; environment variables take precedence because dotenv does not override already-set values by default.

Indexes created at startup

Index	Reason
Unique `employees.emp_code`	Guarantees the employee business code is unique, including concurrent creates.
Unique `attendance_logs(emp_code, date)`	Guarantees at most one daily record for an employee and makes employee/date lookups efficient.
Employee department/code and department/join-date; joined date	Supports department employee listing and headcount filtering.
Attendance date/code; date/late-minutes; employee/punch-in	Supports sorted attendance listing, month-based late scans, and finding a latest punch-in.

> **Seed carefully:** `sample_seed.py` deletes all documents from the two configured collections, then inserts the examples. Run it only against a disposable local database. It is not necessary for the app to start.

If the seed script reports "connection refused" at localhost:27017, Python dependencies are installed but MongoDB is not running or reachable at that address. Start the MongoDB service or configure Atlas first.

4. Attendance request flows

Create employee — `POST /employees`

1. Pydantic validates business code, name/email/department limits, shift formats, distinct shift times, and join date.
2. The route adds server-generated UTC `created_at` and inserts the employee.
3. The unique employee-code index makes concurrent duplicates safe; duplicate inserts return HTTP 409.
4. The response converts `created_at` to epoch milliseconds and hides MongoDB internals.

Punch in — `POST /attendance/punch-in`

1. Validate status (PRESENT, WFH, ON_DUTY) and optional strict epoch-millisecond timestamp.
2. Find employee or return 404. Convert timestamp to UTC and truncate fractional seconds.
3. Convert the instant to IST and choose the attendance date. For overnight shifts, an instant before shift-end belongs to the previous shift-start date.
4. Compute late minutes. Grace is exactly ten minutes; lateness begins strictly after that, and is measured from shift start.
5. Insert a new open record. Unique employee/date index allows one simultaneous duplicate request; loser returns 409.

Punch out — `POST /attendance/punch-out`

1. Find employee, then the latest record whose punch-in is at or before punch-out instant, even across midnight.
2. Return 404 if none exists; 409 if the record is already closed; 422 if time is not after punch-in or exceeds 24 hours.
3. Calculate work hours (2-decimal half-up), overtime (whole minutes and only if at least 30), and half-day (`work_hours < 4.50`).
4. Conditionally update only a record still open. In a race, one succeeds and the other gets 409.

Manual correction — `PATCH /attendance/{emp_code}/{date}`

1. Find employee and its record by business key and attendance date; otherwise return 404.
2. Merge omitted fields with stored values. ABSENT/LEAVE clears punch times; a present status requires punch-in.
3. Validate corrected punch-in belongs to this attendance date and punch-out is after it, at most 24 hours later.
4. Recompute all derived fields. Compare before/after values to record only fields that changed.
5. Append exactly one audit entry with server time, `regularized_by`, reason, and changes.
6. The conditional update detects stale data; a competing correction returns 409 instead of overwriting another history entry.

Normal punches do not write to the manual-correction audit history.

5. Read endpoints and analytics

Endpoint	What happens
`GET /employees`	Optional exact department filter; count only filtered documents; sort by `emp_code`; page in MongoDB.
`GET /attendance`	Optional employee/date/status filters; validate date order; count filtered results; sort date descending then code ascending; paginate in MongoDB.
`GET /analytics/employees/{emp_code}/monthly`	Employee-started pipeline looks up that employee's month logs. It calculates weekdays from the later of join date/month start and reads stored presence, leave, late, and overtime values.
`GET /analytics/departments/summary`	Starts from employees who joined by month-end, then looks up logs. Thus employees with zero logs still count in headcount. It aggregates present weekdays, average work hours over worked records, late totals, leave and ON_DUTY counts.
`GET /analytics/leaderboard/late`	Filters to a month, groups by employee, joins employee details, optionally filters department, then applies MongoDB `$rank`. The rank cutoff is applied after ranking, so ties at the limit are all returned.
`GET /analytics/departments/{department}/trend`	Validates a maximum 92-day range and department existence. A MongoDB aggregation uses `$documents` and `$densify` to make a daily series, looks up headcount/logs per day, then uses `$setWindowFields` for a seven-row moving average.
`GET /admin/explain/{endpoint}`	Rebuilds the matching list query or analytics pipeline and asks MongoDB for its `executionStats` explain plan. Required parameters depend on the endpoint name.

Shared analytics details

- Presence statuses: PRESENT, WFH, ON_DUTY. A half day counts as 0.5; only Monday-Friday attendance adds to monthly present days.
- Late/overtime totals include weekend records; late count is number of records with a positive stored late value.
- Department average hours is an average across qualifying records—not an average of employee averages.
- Reported numbers use half-up rounding. Trend rates and their moving average use four decimal places.
- Attendance trend rates are null on weekends or when headcount is zero; a working day with headcount and no logs has rate zero.

6. Rules the code has to preserve

Rule	Meaning in this application
IST date / overnight shifts	Convert the instant to IST before selecting the calendar attendance date. An overnight punch before shift-end belongs to the previous shift day; overnight shift end is the following day.
Whole-second instants	Input instants are truncated before storage and calculation. HTTP timestamps are epoch milliseconds; seconds-like values, strings, floats, and out-of-range values are invalid.
Late	Strictly more than 10 minutes late; whole-minute floor measured from shift start.
Overtime	Whole-minute floor after shift end; report only when elapsed overtime is at least 30 minutes.
Work hours / half day	Elapsed seconds divided by 3600, rounded half-up to two decimals. Half day if that rounded value is below 4.50.
Working days	Monday-Friday, no holiday calendar; employee join date limits eligible days.
Headcount	Include employees hired on or before the period end, even if there are no attendance records.
Pagination	Starts at page 1; page size defaults to 20 and cannot exceed 100; total reflects the same filters.

Typical response/error outcomes

- **200:** successful read, punch-out, correction, or health response.
- **201:** employee or attendance punch-in created.
- **404:** employee, record, or department not found.
- **409:** duplicate natural key, already-closed punch, or concurrent conflicting correction.
- **422:** invalid request data, timestamp, date range, or business-rule violation.
- **503:** health check cannot reach MongoDB.

> **Local data versus grader data:** the supplied sample is only a handful of documents. It demonstrates shapes, but doesn't exercise every case (for example, many employees tied on lateness or large datasets). Analytics correctness and actual MongoDB execution plans need to be tested against MongoDB and suitable test data.

7. A practical run-through

1. **Start MongoDB.** Confirm the configured local port or Atlas URI is reachable.
2. **Seed optional sample data.** In a separate command prompt: `.venv\Scripts\python.exe sample_seed.py`. It replaces contents in the two collections, so use a disposable database.
3. **Start the server.** `.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000 --reload`.
4. **Check readiness.** Open `http://127.0.0.1:8000/health`; it should respond with `{"status":"ok"}`.
5. **Explore APIs.** Open `http://127.0.0.1:8000/docs`; try creating an employee, punching in/out, and querying its monthly summary.
6. **Inspect performance.** Use an explain endpoint with the relevant query parameters and inspect whether MongoDB uses indexes.

Example API sequence

```
POST /employees
{
  "emp_code": "EMP1001",
  "name": "Ravi Kumar",
  "email": "ravi@example.com",
  "department": "Engineering",
  "shift_start": "09:30",
  "shift_end": "18:30",
  "joined_on": "2026-10-01"
}

POST /attendance/punch-in
{ "emp_code": "EMP1001", "status": "PRESENT" }

POST /attendance/punch-out
{ "emp_code": "EMP1001" }

GET /analytics/employees/EMP1001/monthly?month=2026-10
```

> **Short mental model:** main.py handles HTTP; schemas reject invalid input; attendance.py owns time rules and response conversion; analytics.py describes MongoDB work; database.py connects and indexes. MongoDB stores the source documents and executes filters, writes, joins, grouping, ranking, and trend calculations.

This guide documents the current project structure and behavior as inspected on 9 October 2026.